```
str="Learning Python is very easy"
print(str[24:28],[])
```

```
easy []
```

```
len(str)
```

```
28
```

```
str1 = input ("Enter a string:")
rev=""
for i in str1:
  rev=i+rev
print("Reversed string:",rev)
```

```
Enter a string:Wimbledon
Reversed string: nodelbmiW
```

```
list1=[1,2,3,5,4,2,1,5,2]
list2=[]
for i in list1:
  if i not in list2:
    list2.append(i)
print("List after removing duplicates:",list2)
```

```
List after removing duplicates: [1, 2, 3, 5, 4]
```

```
num=123
a=num%10
b=(num//10)%10
c=num//100
rev=a*100+b*10+c
print(rev)
```

```
321
```

- Object oriented features class object encapsulation inheritance polymorphism abstraction exception handling dynamic binding message passing.
- what is meant by tightly encsapsulation in python.
- what is meant by exception handling. There are two types of errors compile and runtime.

```
a=10
print(a)
print(type(a))
```

```
10
<class 'int'>
```

```
class student:
  def show(Self):
    print("I am in student class")
obj=student()
obj.show()
student().show()
```

```
I am in student class
I am in student class
```

```
class student:
  def __init__(self):
    print("I am in default constructor")
  def show(self):
    print("I am in student class")
obj=student()
obj.show()
```

```
I am in default constructor
I am in student class
```

```
class student:
  def __init__(self):
    print("I am a default constructor")
    print("Hello")
  def show(self):
    print("I am in student class")
  obj=student()
  obj.show()
```

```
I am in default constructor
Hello
I am in student class
```

```
class student:
  def __init__(self):
    self.name="Asin"
    self.age=24
    self.percentage=90
  def showinfo(self):
    print("name:",self.name)
    print("age:",self.age)
    print("percentage:",self.percentage)
s=student()
s.showinfo()
```

```
name: Asin
age: 24
percentage: 90
```

```
class student:
  def __init__(self,name,age,percentage):
    self.name=name
    self.age=age
    self.percentage=percentage
  def showInfo(self):
    print("Name:",self.name)
    print("Age:",self.age)
    print("Percentage:",self.percentage)
s=student("Joy",22,55)
s1=student("Aditya",22,55)
s.showInfo()
s1.showInfo()
```

```
Name: Joy
Age: 22
Percentage: 55
Name: Aditya
Age: 22
Percentage: 55
```

```
# class with parameters.
class student:
  def __init__(self, name, age, percentage):
    self.name = name
    self.age = age
    self.percentage = percentage

  def showinfo(self):
    print("name:", self.name)
    print("age:", self.age)
    print("percentage:", self.percentage)
s1 = student("Astha", 19, 80)
s1.showinfo()

s2 = student("Anamika", 19, 70)
s2.showinfo()
```

```
name: Astha
age: 19
percentage: 80
name: Anamika
age: 19
percentage: 70
```

```
# Taking user input.
class student:
  def __init__(self):
    self.name = input("name:")
    self.age = input("age:")
    self.percentage = input("percentage:")
  def showinfo(self):
    print("name:", self.name)
    print("age:", self.age)
    print("percentage:", self.percentage)
s1 = student()
s1.showinfo()

s2 = student()
s2.showinfo()
```

```
name:Yamini
age:23
percentage:89
name: Yamini
age: 23
percentage: 89
name:Wamiqa
age:23
percentage:78
name: Wamiqa
age: 23
percentage: 78
```

```
# Taking User input.
class student:
  def __init__(self, name, age, percentage):
    self.name = name
    self.age = age
    self.percentage = percentage

  def showinfo(self):
    print("name:", self.name)
    print("age:", self.age)
    print("percentage:", self.percentage)
name=input("Enter name:")
age=input("Enter age:")
percentage=input("Enter percentage:")
s1=student(name,age,percentage)
s1.showinfo()
```

```
Enter name:Rasha
Enter age:17
Enter percentage:89
name: Rasha
age: 17
percentage: 89
```

```
# Python does not support function overloading, if such a case occurs, the last function is executed.
class MyPython:
  def __init__(Self):
    print("First Constructor")
  def __init__(Self):
    print("Second constructor")
m=MyPython()
```

```
Second constructor
```

```
class MyPython:
  def __init__(self):
    print("Constructor is called")
  def __del__(self):
    print("Destructor is called")
  def func(self):
    print("Func code execution")
m1=MyPython()
m1.func()
```

```
Constructor is called
Func code execution
```

- Variable created using self is called instance variable.

```
# delete instance variable
class mypython:
  def __init__(self):
    self.a=10
    self.b=20
    self.c=30
    self.d=40
  def func(self):
     del self.d
m=mypython()
print(m.__dict__)
m.func()
print(m.__dict__)
del m.c
print(m.__dict__)
```

```
{'a': 10, 'b': 20, 'c': 30, 'd': 40}
{'a': 10, 'b': 20, 'c': 30}
{'a': 10, 'b': 20}
```

```
# destructor
class MyPython:
  def __init__(self):
    print("Constructor is called")
  def __del__(self):
    print("Destructor is called")
  def func(self):
    print("Function code is executed")
m=MyPython()
m.func()
```

```
Constructor is called
Function code is executed
```

```
# Static variable
class MyPython:
  x=20
  def __init__(self):
    self.y=10
m1=MyPython()
m2=MyPython()
print("m1 -->",m1.x,m1.y)
print("m2 -->",m2.x,m2.y)
MyPython.x=111
m1.y=222
print("m1 -->",m1.x,m1.y)
print("m2 -->",m2.x,m2.y)
```

```
Destructor is called
m1 --> 20 10
m2 --> 20 10
m1 --> 111 222
m2 --> 111 10
```

```
# Local variables
class Test:
  def func1(self):
    a=100
    print(a)
  def func2(self):
    b=222
    print(b)
    print(a)
t=Test()
t.func1()
t.func2()
```

```
100
222
```

```
10
```

```
# Local variables
class Test:
  def func1(self):
    a=100
    print(a)
  def func2(self):
    a=11
    b=122
    print(b)
    print(a)
t=Test()
t.func1()
t.func2()
```

```
100
122
11
```

- Accept 5 paper marks, rollno, name. Calculate total and percentage. If percentage>=85,, student can take admission else can't take. Use CLASS, OBJECT, CONSTRUCTOR

```
class Student:
    def __init__(self):
        self.name = input("Enter student name: ")
        self.rollno = int(input("Enter roll number: "))
        print("\nEnter marks for 5 subjects (out of 100 each):")
        self.p1 = float(input("Paper 1: "))
        self.p2 = float(input("Paper 2: "))
        self.p3 = float(input("Paper 3: "))
        self.p4 = float(input("Paper 4: "))
        self.p5 = float(input("Paper 5: "))

    def calculate_and_check(self):
        self.total = self.p1 + self.p2 + self.p3 + self.p4 + self.p5
        self.percentage = (self.total / 500) * 100

        print("\n" + "=" * 30)
        print("--- STUDENT DETAILS ---")
        print(f"Name       : {self.name}")
        print(f"Roll No    : {self.rollno}")
        print(f"Total Marks: {self.total} / 500")
        print(f"Percentage : {self.percentage:.2f}%")
        print("-" * 30)

        if self.percentage >= 85:
            print("Result     : Student CAN take admission.")
        else:
            print("Result     : Student CANNOT take admission.")
        print("=" * 30)
s1 = Student()
s1.calculate_and_check()
```

```
Enter student name: Rasha
Enter roll number: 78

Enter marks for 5 subjects (out of 100 each):
Paper 1: 45
Paper 2: 78
Paper 3: 90
Paper 4: 99
Paper 5: 89

==============================
--- STUDENT DETAILS ---
Name       : Rasha
Roll No    : 78
Total Marks: 401.0 / 500
Percentage : 80.20%
------------------------------
Result     : Student CANNOT take admission.
==============================
```

```
class A:
  def showA(self):
```

```
    print("showA function")
class B(A):
  def showB(self):
    print("showB function")
obj=B()
obj.showA()
obj.showB()
```

```
showA function
showB function
```

Start coding or generate with AI.

```
# Multilevel Heritance
class A:

    def showA(self):
        print("showA function")

class B(A):
    def showB(self):
        print("showB function")

class C(B):
    def showC(self):
        print("showC function")



obj = C()
obj.showA()
obj.showB()
obj.showC()
```

```
showA function
showB function
showC function
```

```
# Multiple Inheriatnce
class A:
    def showA(self):
        print("showA function")

class B:
    def showB(self):
        print("showB function")

class D:
    def showD(self):
        print("showD function")

    def showC(self):
        print("showC function")

obj = C()
obj.showA()
obj.showB()
obj.showC()
obj.showD()
```

```
# Multiple Inheritance
class A:
  def showA(self):
    print("showA function")
class B:
  def showB(self):
    print("ShowB function")
class C(B,A):
  def showC(self):
    print("showC function")
obj=C()
obj.showC()
obj.showB()
obj.showA()
```

```
showC function
ShowB function
showA function
```

```
#Hierarchical Inheritance
class A:
  def showA(self):
    print("showA function")
class B(A):
  def showB(self):
    print("ShowB function")
class C(A):
  def showC(self):
    print("showC function")
obj=C()
obj.showC()
obj.showA()
```

```
showC function
showA function
```

```
# Hybrid Inheritance
class A:
    def showA(self):
        print("showA function")

class B(A):
    def showB(self):
        print("showB function")
class C():
    def showC(self):
      print("showC function")

class D(B,C):
    def showD(self):
        print("showD function")



obj = D()
obj.showC()
obj.showA()
obj.showD()
obj.showB()
```

```
showC function
showA function
showD function
showB function
```

```
class A:
  def showA(self):
    print("showA function")
class B(A):
  def showB(self):
    print("showB function")
class C():
  def showC(self):
    print("showC function")
class D(B,C):
  def showD(self):
    print("showD function")
obj=D()
obj.showC()
obj.showA()
obj.showD()
obj.showB()
```

```
showC function
showA function
showD function
showB function
```

```
# Function Overriding
```

```
# Function Overriding
class parent:
  def bike(self):
    print("Splendor")
class child:
  def bike(self):
    print("Bullet")
obj=child()
obj.bike()
```

```
Bullet
```